

Déploiement pour plusieurs coopératives

La traçabilité agricole, de la parcelle au marché.

Version du 2026-10-24

Déploiement pour plusieurs coopératives

> **État** : les images Docker n'ont **pas pu être construites dans l'environnement de développement** (pas de démon Docker) ; elles sont construites par la CI. Les offres, tarifs et régions des hébergeurs cités ne sont **pas vérifiés** : à confirmer avant tout achat. Ce guide complète `DEPLOIEMENT.md` (installation d'une instance) et `EXPLOITATION.md` (sauvegardes, supervision).

1. Principe : une plateforme, plusieurs coopératives

AgriTrace est **multi-coopératives par conception**. Une seule instance (une base, une API, un portail) sert N coopératives :

- chaque coopérative est une ligne `organizations` ; toutes les tables portent `organization_id` ;
- l'isolation est faite **par la base** (sécurité par ligne, RLS) : chaque transaction fixe `app.current_org`, et PostgreSQL ne montre que les lignes de cette coopérative, même en cas d'erreur de code ;
- l'API doit tourner avec le rôle non propriétaire `agritrace_app` (créé par `deploy/db/10-roles.sh`). **Avec le propriétaire, l'isolation disparaît** : à vérifier à chaque installation ;
- l'**administrateur de plateforme** (vous) crée les coopératives ; chaque coopérative a ses administrateurs, rôles, caisses, stocks et documents.

Deux architectures possibles, selon la connexion des coopératives :

Architecture	Quand	Principe
A. Tout centralisé (recommandée pour démarrer)	coopératives avec Internet, même intermittent (mobile hors connexion déjà géré)	une instance centrale ; les utilisateurs se connectent au portail
B. Serveur local + central	coopérative à connexion très faible, ou qui veut travailler sans Internet au bureau	un serveur local (`AGRITRACE_MODE=local`) par coopérative, qui envoie ses données au central ; le central est alors en lecture seule pour cette coopérative

On peut **mélanger** : certaines coopératives directement sur le central, d'autres avec un serveur local (voir `LOCAL_CENTRAL_BUREAU.md`).

2. Quel hébergeur pour plusieurs coopératives

Critères : PostgreSQL 16 **avec PostGIS** et droit de créer des rôles ; disque pour les documents (volume chiffré) ; sauvegardes hors site ; proximité des utilisateurs (latence) ; coût prévisible ; vous gardez la main sur les données.

Option	Convient à	Points d'attention
VPS Linux + Docker Compose (Hetzner, OVHcloud, Scaleway ou équivalent)	**1 à ~20 coopératives** : meilleur rapport simplicité / coût / contrôle. **Recommandé.**	vous gérez mises à jour système, pare-feu, sauvegardes
VPS pour l'API + base PostgreSQL gérée (avec PostGIS)	20 coopératives et plus, ou exigence de sauvegarde avec restauration à un instant donné	vérifier PostGIS, création de rôles, `GRANT SET ON PARAMETER session_replication_role` (migration 012), mode du pooler (voir §8)
Serveur dédié ou second VPS	forte charge, ou séparation base / API	séparer base et API sur un réseau privé
Hébergeur africain (région proche des coopératives)	latence et exigences de souveraineté des données	offres à vérifier une par une : Docker, IPv4, sauvegarde, support
Vercel	**portail statique seulement**	ne peut pas héberger l'API (base, fichiers, tâches)
Supabase	**base gérée seulement**, sous réserve du §8	n'héberge pas l'API NestJS

Recommandation : démarrer avec **un VPS Linux + Docker Compose dans la région la plus proche des coopératives**, avec sauvegarde hors site chez un second fournisseur ; passer à une base gérée quand le nombre de coopératives ou l'exigence de reprise l'imposent. Le choix n'est pas définitif : les données se déplacent par `pg_dump` / `pg_restore` et copie du volume des documents.

Dimensionnement indicatif (à mesurer avec `scripts/load-test.js`)

Coopératives	Machine	Disque
1 – 5	2 vCPU, 4 Go	80 Go
5 – 20	4 vCPU, 8 Go	200 Go, extensible
20 +	base et API séparées, base gérée	selon les pièces jointes (photos, PDF)

Le disque est surtout consommé par les **documents** (volume `documents`) et les sauvegardes. Ce sont des ordres de grandeur, pas des garanties.

3. Installation de l'instance centrale

1. Serveur Linux à jour, Docker et Compose, pare-feu ouvrant seulement 22 (clé SSH, pas de mot de passe), 80 et 443.
2. Dépôt cloné, puis `.env` : tous les secrets **aléatoires et propres à cette plateforme** :
   - `JWT_SECRET` (≥ 32 caractères, `openssl rand -base64 48`), `SECRETS_KEY`, `DOCUMENTS_KEY` (`openssl rand -base64 32`) ;
   - `REQUIRE_2FA=true` (obligatoire en pratique avec plusieurs coopératives : un compte compromis a plus d'impact) ;
   - `CORS_ORIGINS` si le portail est sur un autre domaine ; `TRUST_PROXY` selon le nombre de proxys ; `METRICS_TOKEN` pour la supervision.

3. `docker compose -f docker-compose.prod.yml up -d --build` ; vérifier `migrate` terminé sans erreur puis `GET /v1/ready`.
4. **Reverse proxy HTTPS** (Caddy le plus simple : certificat automatique) devant le portail (port 8080 du compose, non exposé directement).
5. Créer l'administrateur de plateforme : `node scripts/create-platform-admin.js` (voir `DEPLOIEMENT.md`), activer sa double authentification **immédiatement**.
6. Ne jamais lancer `seed-demo.js` ni les scripts de démonstration en production.

Domaines

- **Un seul domaine** (ex. `agritrace.exemple.org`) : les utilisateurs de toutes les coopératives s'y connectent ; l'organisation est déterminée par leur compte. Le plus simple, **recommandé**.
- **Un sous-domaine par coopérative** possible au niveau du proxy (même portail derrière), mais l'application n'en dépend pas : c'est un confort de marque, pas d'isolation. L'isolation reste celle de la base.

4. Ouvrir une coopérative (liste à cocher)

Faite par l'administrateur de plateforme, dans la console centrale (

Ajouter une coopérative

) ou par l'API :

1. `POST /v1/organizations` : code court unique, nom, pays → crée la coopérative et son rôle `coop_admin` ;
2. `POST /v1/organizations/:id/admins` : e-mail, nom, mot de passe fort → premier administrateur de la coopérative ;
3. transmettre l'identifiant **et** le mot de passe par deux canaux différents ; l'administrateur change son mot de passe et active sa 2FA à la première connexion ;
4. l'administrateur de la coopérative crée ses rôles, utilisateurs, zones, producteurs (voir le guide de formation) ;
5. **vérifier l'isolation** : se connecter avec un compte de cette coopérative et constater qu'aucune donnée d'une autre n'apparaît (listes, exports, documents) ;
6. si la coopérative est en architecture B : générer le **code de jumelage** (console → *Nouveau code*) et l'entrer sur le serveur local ;
7. consigner : date d'ouverture, administrateur, version, responsable de la coopérative.

Fermeture ou suspension : suspendre l'organisation (les comptes ne peuvent plus se connecter), exporter ses données, puis ne supprimer qu'après accord écrit et sauvegarde vérifiée.

5. Sauvegardes (point critique du multi-coopératives)

Une panne ou une perte touche **toutes** les coopératives à la fois.

- `scripts/backup.sh` chaque nuit : base + volume des documents, avec `BACKUP_PASSPHRASE_FILE` (chiffrement AES-256) ;
- **copie hors du serveur et chez un autre fournisseur** (stockage objet chiffré) ; la phrase de chiffrement, `DOCUMENTS_KEY`, `SECRETS_KEY` et `JWT_SECRET` sont conservés **à part**, jamais sur le même support que les sauvegardes ;
- `scripts/restore-test.sh` au moins chaque mois, sur une machine jetable ;
- rétention : 30 jours en ligne, plus long (mensuel) hors site, selon les obligations de la coopérative ;
- restauration **d'une seule coopérative** : non fournie en un clic. Restaurer la sauvegarde dans une base jetable, puis extraire les lignes de l'organisation concernée. À prévoir et à tester avant d'en avoir besoin ;
- objectifs proposés : RPO 24 h, RTO quelques heures ; un RPO plus court exige l'archivage WAL ou une base gérée avec restauration à un instant donné.

6. Supervision

- `GET /v1/health` (vivacité), `GET /v1/ready` (base), `GET /v1/metrics` avec `METRICS_TOKEN` ;
- surveiller : espace disque (documents + sauvegardes), connexions à la base, retard de sauvegarde, échecs de connexion répétés, certificat HTTPS ;
- alerte e-mail/SMS sur indisponibilité (service externe de sondage) ;
- `LOG_REQUESTS=true` : journal d'accès sans corps ni jeton ; journal d'audit applicatif par coopérative.

7. Mises à jour

1. Prévenir les coopératives (plage horaire creuse) ; sauvegarder ; copie de la base dans un environnement de test et essai de la migration ;
2. `git pull && docker compose -f docker-compose.prod.yml up -d --build` : `migrate` s'applique avant l'API ;
3. contrôler `/v1/ready`, une connexion par coopérative-test, une écriture ;
4. architecture B : le serveur local et le central doivent avoir la **même version de schéma** ; mettre à jour le central **puis** les serveurs locaux, dans la même fenêtre.

Retour arrière : restaurer la sauvegarde d'avant mise à jour (les migrations sont additives, mais l'ancien code n'ignore pas toujours les nouvelles colonnes).

8. Si vous choisissez une base gérée (dont Supabase)

À **vérifier avant de s'engager**, sur un essai :

- PostGIS disponible, PostgreSQL 16 ou compatible ;
- création des rôles `agritrace_owner` et `agritrace_app` (non superutilisateur) ;
- `GRANT SET ON PARAMETER session_replication_role` pour le rôle applicatif (réception des lots de réplication) — peut être refusé sur certains services ;

- **pooler en mode transaction** : l'application fixe `app.current_org` par transaction (`set_config(..., true)`), compatible en principe, mais à tester ; préférer la connexion directe pour l'API et réserver le pooler aux outils ;
- région, chiffrement au repos, sauvegardes et durée de rétention, coût selon la taille ;
- les **documents** restent à stocker ailleurs (volume de l'API ou stockage objet) : l'application les écrit aujourd'hui dans `DOCUMENTS_DIR`.

9. Protection des données

- chaque coopérative reste responsable de ses données personnelles (producteurs, paiements) ; la plateforme en est le sous-traitant : prévoir un **contrat d'hébergement** précisant finalité, durée de conservation, sauvegardes, sous-traitants, notification d'incident ;
- vérifier les règles du pays (localisation des données, autorité de protection) et la région de l'hébergeur ;
- accès administrateur de plateforme : peu de personnes, 2FA obligatoire, journalisé ; ne pas consulter les données d'une coopérative sans demande ;
- les sauvegardes contiennent tout : droits `600`, chiffrées, accès restreint.

10. Passage à l'échelle

- tant que la charge est faible : un seul VPS ;
- ensuite : séparer la base (réseau privé), puis base gérée, puis plusieurs API derrière un équilibreur (l'API est sans état, sauf le dossier des documents → stockage partagé ou objet nécessaire) ;
- mesurer avant d'agrandir : `scripts/load-test.js`, métriques, temps de réponse par coopérative.

11. Coûts à prévoir (sans chiffres, à établir avec devis)

Serveur(s), stockage des sauvegardes hors site, nom de domaine, supervision externe, temps d'exploitation (mises à jour, sauvegardes, support). Définir qui paie quoi entre la plateforme et les coopératives **avant** la première ouverture.